



Sesión L09- Streams

- 00. [Preparamos el directorio de trabajo](#)
- 01. [Proyecto facturas](#)
- 02. [Uso de la clase Console](#)
- 03. [Uso de streams basados en caracteres](#)
- 04. [Uso de streams basados en bytes](#)

[Recuerda!](#)

00

Preparamos el directorio de trabajo



Para realizar esta práctica vamos a crear el directorio *L09-Streams*, dentro de nuestro directorio de trabajo. Por lo tanto tendremos que hacer:

```
> cd Practicas  
> mkdir L09-Streams  
> cd L09-Streams
```

Este será el subdirectorio en el que vamos a trabajar durante el resto de la sesión. Por lo tanto cualquier fichero o directorio que creemos en esta sesión estará dentro de *L09-Streams*.

01

Proyecto facturas



Crea un nuevo proyecto Maven **L09-facturas** en tu directorio de trabajo (*L09-Streams*). Recuerda que hemos explicado cómo hacerlo en la práctica L05

El *Group Id* del proyecto será **pa**, el *Artifact Id*: **L09-facturas**, y el valor de *packaging* será **jar**. Recuerda modificar el fichero **pom.xml** del proyecto y comprobar que en las propiedades del proyecto se indica que el nivel de código fuente es 21.

Crea una *Run Configuration* con nombre **L09-facturas-run** a la que asociaremos los comandos:

```
clean compile exec:java -Dexec.mainClass=pa.Facturacion
```

Ya deberías saber qué hace cada comando maven de las *run configurations*.

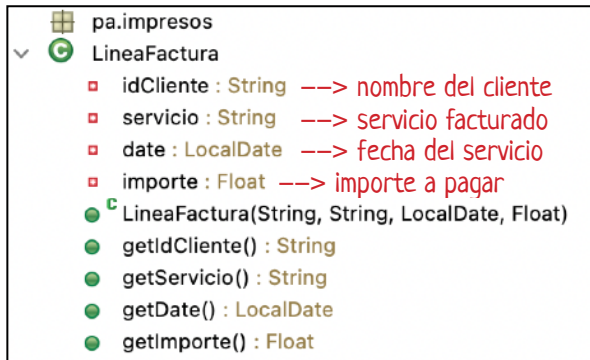
Queremos gestionar el proceso de crear facturas a partir de líneas de facturas, de forma que podamos imprimir y leer las facturas tanto desde pantalla como en ficheros (de texto y binarios).



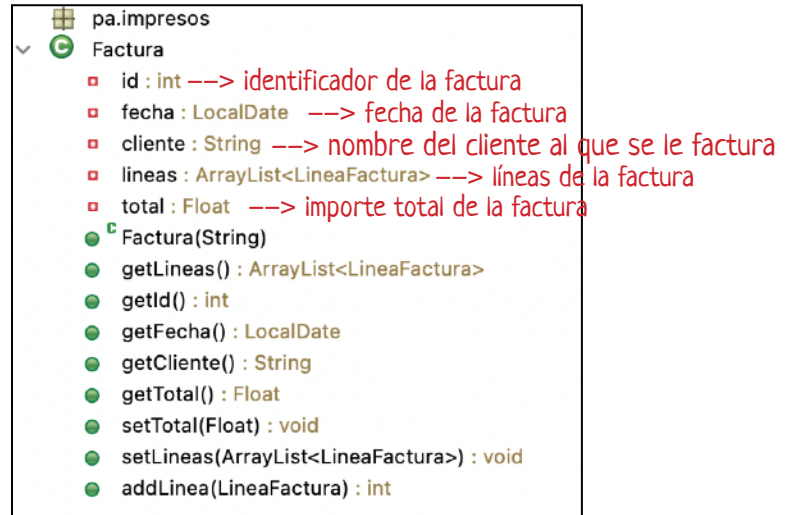
Vamos a trabajar con dos paquetes:

- **pa:** aquí tendremos de momento la clase Facturacion.
- **pa.impresos:** contiene las clases Factura y LineaFactura.

Comenzamos implementando las clases del paquete **pa.impresos**. A continuación mostramos la descripción abstracta de dichas clases.



Clase **LineaFactura**



Clase **Factura**

- **LineaFactura:** sus campos son privados.

Podemos crear un objeto de tipo LineaFactura a partir del nombre del cliente, el servicio facturado, la fecha del servicio y el importe a cobrar (en ese orden)

Esta clase contiene únicamente **getters**, puedes crearlos automáticamente desde eclipse a partir del menú contextual del editor con la opción **Source**, como ya hemos hecho en otras ocasiones

- **Factura:** el campo **id** contiene el identificador de la factura, que es único para todas ellas.

Podemos crear un objeto de tipo Factura a partir del nombre del cliente al que se le va a asociar dicha factura. Cuando creamos cada factura, se le asocia la fecha actual (se obtiene invocando al método estático `Localdate.now()`), un importe total con valor 0.0 y una lista vacía de objetos LineaFactura.

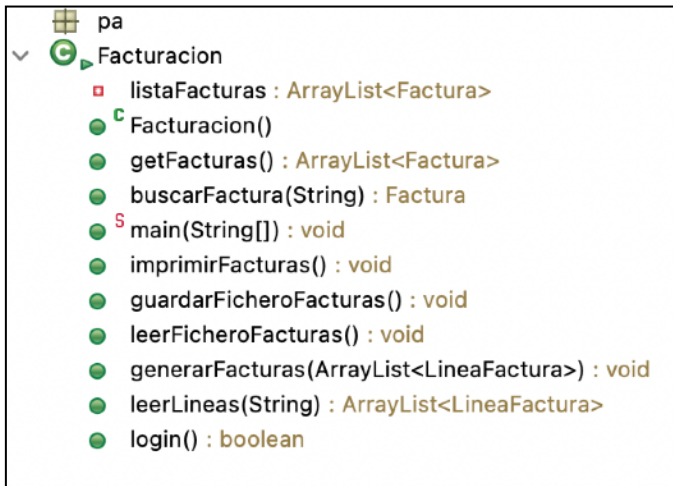
Además, cuando creamos cada factura, se le asigna su valor de "id". Usaremos un atributo estático, al que llamaremos "ultimoID" para garantizar que cada factura tendrá un identificador diferente, el cual será un valor numérico que se irá incrementando en 1 para cada factura. La primera factura creada tendrá asociado un id = 1.

Cada factura sólo puede contener un máximo de 6 "lineas" (tamaño del atributo "lineas"). Debes crear un atributo adicional para guardar dicha información (en la imagen no aparece dicho atributo porque sólo hemos mostrado los atributos privados).

La clase Factura implementa **dos setters** y varios **getters**, que puedes generar de forma automática.

El método **addLinea()** recibe como parámetro un objeto de tipo LineaFactura y lo añade a la lista "lineas" (siempre y cuando no supere el número máximo de líneas). Cuando añadimos una línea a la factura, se incrementa el campo "total" con el importe de dicha línea. El método **addLinea()** devuelve el número de línea añadido, o el valor -1 si la factura ya tiene 6 líneas.

Ahora vamos con a crear clase **pa.Facturacion**. Mostramos su descripción abstracta en la siguiente imagen. Vamos a implementar todos sus métodos, que iremos detallando a continuación.



Fíjate que esta clase tiene un método **main()**, a partir del cual ejecutaremos nuestra aplicación.

En esta ocasión no hemos usado una clase adicional para probar nuestro código, y por lo tanto, en nuestro método **main()** tendremos que crear un objeto de tipo **Facturacion** para poder invocar al resto de métodos (no estáticos) de la clase.

La clase **Facturacion** sólo tiene un atributo privado con una lista de **Facturas**, con un único método getter (**getFacturas()**)

Clase **Facturacion**

El constructor de la clase **Facturación** inicializa la lista de facturas a una lista vacía.

Métodos a implementar:

■ **buscarFactura()**: recibe como parámetro el nombre del cliente de la factura a buscar. El método busca la **Factura** en la lista de facturas cuyo atributo "cliente" coincida con el valor del parámetro. El método devuelve la **Factura** correspondiente de la lista "listaFacturas", o el valor null si dicha factura no está en la lista.

■ **generarFactura()**: recibe como parámetro una lista de objetos **ListaFactura**.

El método crea las facturas a partir de la lista pasada por parámetro (una factura para cada cliente diferente). Cada línea de factura tiene asociado un nombre de cliente (idCliente), de forma que cada factura tiene hasta 6 líneas de ese cliente (atributo "cliente" del objeto **Factura**). Si en la lista de objetos **ListaFactura** hay más de 6 líneas para un mismo cliente, sólo se usarán las 6 primeras para crear la factura, y el resto simplemente se ignorarán.

Este método actualiza el atributo "listaFacturas" creando nuevas facturas, o bien modificando las existentes añadiéndoles más líneas (siempre que no sobrepasen el máximo de 6 líneas por factura).

Por ejemplo, suponiendo que la lista de facturas está inicialmente vacía, y pasamos como parámetro una lista de 10 líneas de facturas, de las cuales 3 llevan el nombre "x" y 7 llevan el nombre "y", el método añadirá dos nuevas facturas a "listaFacturas": una tendrá tres líneas (asociadas a "x") y la otra tendrá seis líneas (asociadas a "y", la séptima la ignoramos).



02

Uso de la clase Console



Implementa el método **login()** de la clase Facturacion.

Dicho método usará la clase Console para solicitar un login y password al usuario (en las transparencias tienes un ejemplo de uso de dicha clase).

De momento sólo permitiremos el acceso al usuario cuyo login es **"usuario"** y su contraseña es **"factura"**.

Si estamos ejecutando el programa desde el terminal:

El método solicitará al usuario sus credenciales y los leerá. Si el usuario introduce sus credenciales correctamente, el método imprimirá en pantalla el mensaje "Credenciales válidos". Si el login o password no son los correctos, imprimirá el mensaje "Login o password inválido".

Si no estamos ejecutando el programa desde el terminal, el valor del objeto Console sera **null**, en cuyo caso el método no hará nada.

Recuerda que no podrás usar la clase Console si ejecutas el programa desde el Eclipse. No es mayor problema ya que siempre podemos ejecutar nuestro código desde el terminal usando el comando maven desde la carpeta que contiene el fichero pom.xml:

```
mvn exec:java -Dexec.mainClass=pa.Facturacion
```

Añade el código necesario en el método **main()** de la clase Facturación para ejecutar el método **login()** que acabas de implementar. Recuerda que primero debes asegurarte de que existe el objeto de tipo Console (mira el ejemplo de las transparencias). Si no existe el objeto de tipo Console imprimiremos el mensaje "Debe ejecutar la aplicación desde el terminal".

Si existe el objeto Console, invocaremos al método **login()** de forma que permitiremos que el usuario realice un máximo de 3 intentos para introducir sus credenciales correctos.

Si el usuario agota los tres intentos, se mostrará el mensaje "Agotado el número de intentos". En caso contrario mostraremos el mensaje "Bienvenido."

A continuación mostramos tres ejemplos de ejecución de nuestro método **main()** que, de momento, sólo usa el método **login()**.

```
[INFO] --- exec-maven-plugin:1.6.0:java
(default-cli) @ facturas-L09 ---
Ejecute el programa desde el terminal
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

Ejecución desde Eclipse

```
Login: otro
Contraseña:
Login o password inválido

Login: usuario
Contraseña:
Credenciales válidos
Bienvenido
```

Ejecución desde el terminal.

Credenciales correctos en el segundo intento

```
Login: ppp
Contraseña:
Login o password inválido
```

```
Login: jeje
Contraseña:
Login o password inválido
```

```
Login: 87yt
Contraseña:
Login o password inválido
Agotado el número de intentos
```

Ejecución desde el terminal.

Agotamos los intentos



03

Uso de *streams* basados en caracteres

En el directorio **L09-plantillas** tenéis el fichero **datosLineas.txt**. Es un fichero de texto que contiene la información de varias líneas de facturas. Copia el fichero en el directorio `src/main/resources` de tu proyecto maven.

Puedes abrir dicho fichero en el editor de Eclipse para ver su contenido (pero no lo modifiques!!!).

Vamos a seguir implementando los métodos de la clase **Facturacion**.

Implementa el método `leerLineas()`.

El método recibe como parámetro el nombre del fichero con la información de las líneas de factura (`src/main/resources/datosLineas.txt`).

El método lee el contenido de dicho fichero y devuelve una lista de objetos *LineaFactura*.

Usa la clase **FileReader** combinada con la clase **BufferedReader**.

En este caso vamos a leer línea por línea usando el método `readline()` de la clase **BufferedReader**. Consulta su documentación javadoc para ver cuál es su signatura y qué hace el método.

(ver [https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/BufferedReader.html#readLine\(\)](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/BufferedReader.html#readLine()))

Dado que los atributos fecha y precio son de tipo *LocalDate* y *float*, respectivamente, obtendremos dichos valores a partir de la cadena de caracteres que hemos leído del fichero usando los métodos estáticos **LocalDate.parse(CharSequence)** y **Float.parseFloat(String)**.

Consulta su documentación javadoc para usarlos correctamente:

(ver <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/LocalDate.html> y

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Float.html>)

Para comprobar que lees los datos correctamente puedes imprimir las líneas por pantalla a medida que las vas leyendo.

Implementa el método `imprimirFacturas()`

Dicho método genera un fichero de texto para cada factura de la lista de facturas de la clase **Facturacion**.

Cada fichero se generará en el directorio raíz del proyecto (donde está el fichero `pom.xml`), por lo que no tendrás que añadir ningún prefijo al nombre de fichero, y su nombre será **fichero<X>.txt**, siendo **<X>** el valor del atributo **id** de cada factura

Debes usar las clases **FileWriter** y **BufferedWriter**.

En este caso vamos a escribir cadenas de caracteres en el fichero, línea por línea usando el método: **write(String)**. Usa también el método **newline()** para escribir un retorno de carro en el fichero.

Usa el método **toString()** de la clase *Localdate*, para obtener la representación de la fecha en forma de cadena de caracteres.

A continuación mostramos el contenido de `factura1.txt` y `factura2.txt`. Estos ficheros son generados por el método `imprimirFacturas()` teniendo en cuenta los datos de las facturas del fichero **datosLineas.txt** proporcionado.

Id factura: 1 Fecha: 2023-11-25		
Cliente: Pedro Garcia		
Fecha	Servicio	Importe
2023-09-21	operacion	300.75
2023-09-25	habitacion	200.75
2023-09-21	anestesia	500.0
Total importe: 1001.5 euros		

Contenido del fichero **factura1.txt**

Id <u>f</u> actura: 2 <u>F</u> echa: 2023-11-25		
Cliente: Gloria Martinez		
Fecha	Servicio	Importe
2023-04-18	radiografia	550.15
2023-09-21	ecografia	440.75
2023-03-14	analitica	125.75
2023-04-18	radiografia	550.15
2023-09-21	ecografia	440.75
2023-03-14	densitometria	125.75
Total importe: 2233.3 euros		

Contenido del fichero **factura2.txt**

Para que la información de cada línea imprima en columnas con la misma anchura, vamos a formatear la salida escribiendo en el *buffer* las siguientes cadenas de caracteres:

```
//Cabecera columnas
String.format("%-14s%-16s%-10s", "Fecha", "Servicio", "Importe")

//línea de separación
String.format("%-14s%-16s%-10s", "-", "-", "-").replace(' ', '-')

//Para cada línea de factura imprimiremos
String.format("%-14s", <fecha_en_formato_cadena>));
String.format("%-16s", <servicio>));
String.format("%-10s", <importe>));
```

En el método **main()** de la clase **Facturación**, después de habernos identificado correctamente añade el código necesario para leer las líneas de facturación del fichero de texto, generar las facturas e imprimirlas.



04

Uso de *streams* basados en bytes

Implementa el método **guardarFicheroFacturas()**

Dicho método genera el fichero **facturas.dat**, que contiene la lista de facturas de la clase **Facturacion**. Es un fichero de *bytes* (no es legible si lo abrimos en el editor de eclipse). Igual que antes, no hay que añadir ningún prefijo a dicho nombre, éste se generará en la carpeta del proyecto.

Usaremos la clase **ObjectOutputStream** combinada con **BufferedOutputStream** y **FileOutputStream**, para escribir la información en el fichero usando directamente objetos java.

Tienes un ejemplo de código en las transparencias.

La clase **ObjectOutputStream**, a partir de los objetos java, debe "convertirlos" en una secuencia de bytes. A este proceso se le denomina "serialización" y para poder realizar dicho proceso, las clases de los objetos que van a ser "escritos" en el fichero tienen que ser "serializables" (implementar la interfaz **Serializable**).



Por lo tanto, tendremos que cambiar la implementación de las clases *Factura* y *LineaFactura* de forma que implementen la interfaz **Serializable**. Esta interfaz no tiene métodos ni atributos, simplemente sirve para identificar la semántica de que ese clase puede ser "serializada". (Ver <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/Serializable.html>)

Deberás añadir el atributo `serialVersionUID`, puedes inicializarlo con cualquier valor, por ejemplo:

```
private static final long serialVersionUID = 1L;
```

Este atributo es conveniente declararlo ya que cualquier clase "serializable" podrá ser "serializada" (pasará de objeto java a bytes) y también "deserializada" (convertir bytes a objetos java, esto lo haremos cuando tengamos que leer objetos java a partir del fichero de bytes). El atributo `serialVersionUID` asigna un número al objeto serializado basado en su contenido, de forma que cuando éste se deserialice, se comprobará que el número asociado sea el mismo (en el enlace anterior de la interfaz `Serializable`, tenéis descrito el significado y uso del atributo `serialVersionUID`).

Para escribir los objetos java en el fichero usaremos el método `writeObject()` (de la clase `OutputStream`)

El método ***guardarFicheroFacturas()*** inicialmente imprimirá por pantalla el mensaje **"GUARDANDO FACTURAS"**, y cada vez que escriba un objeto *Factura* en el fichero imprimirá el mensaje **"Guardada factura <idFactura>"** (siendo **<idFactura>** el valor del atributo **id**).

Para probar el método invócalo en el *main()* después del método *imprimirFacturas()* del ejercicio anterior.

Implementa el método ***leerFicheroFacturas()***

Este método lee el contenido del fichero "facturas.dat" e imprime por pantalla los datos leídos.

Usaremos la clase *ObjectInputStream* combinada con *BufferedInputStream* y *FileInputStream*, para leer la información del fichero usando directamente objetos java.

Para probar el método invócalo en el *main()* después del método ***guardarFicheroFacturas()***

Mostramos un **ejemplo de ejecución** del método *leerFicheroFacturas()* (no es necesario que formatees de ninguna forma especial la salida por pantalla).

```
FACTURA LEIDA:
Fecha: 2023-11-26
Cliente: Pedro Garcia
Total factura: 1001.5 euros
Número de líneas: 3
  Servicio: operacion    2023-09-21    300.75
  Servicio: habitacion   2023-09-25    200.75
  Servicio: anestesia    2023-09-21    500.0
FACTURA LEIDA:
Fecha: 2023-11-26
Cliente: Gloria Martinez
Total factura: 2233.3 euros
Número de líneas: 6
  Servicio: radiografia  2023-04-18    550.15
  Servicio: ecografia    2023-09-21    440.75
  Servicio: analitica    2023-03-14    125.75
  Servicio: radiografia  2023-04-18    550.15
  Servicio: ecografia    2023-09-21    440.75
  Servicio: densitometria 2023-03-14    125.75
```

Ejecución del método ***leerFicheroFacturas()***

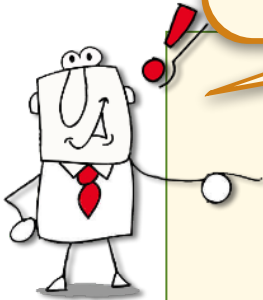
Recuerda subir a GitHub el trabajo realizado hasta el momento:

```
> git add . //desde el directorio que contiene el repositorio git
> git commit -m"Terminada la práctica L09-streams"
> git push
```

RECUERDA



¿Qué **conceptos** y **cuestiones** me deben quedar CLAROS después de hacer la práctica?



- Qué es y para qué sirve un stream.
- Podemos clasificar los streams en función de si están basados en bytes o en caracteres.
- El objeto de tipo Console es único y nos permite leer contraseñas del teclado sin que éstas sean visibles.
- Los streams pueden combinarse, esto nos permite hacer más eficiente la lectura/escritura de datos (por ejemplo usando un Buffered stream), o facilitar la programación (usando por ejemplo un Data stream o un Object stream).
- Siempre que "abramos" un stream tenemos que "cerrarlo". Si usamos un try-with-resources, no necesitaremos usar el método close() de forma explícita
- Todas las operaciones de lectura/escritura lanzan excepciones que deberemos capturar (excepto los métodos de la clase PrintStream).
- Cada stream contiene datos que "fluyen" siempre en un mismo sentido (o es un stream de entrada o es de salida).